EASTERN INTERNATIONAL UNIVERSITY
SCHOOL OF COMPUTING AND INFORMATION TECHNOLOGY

Practice Assignment – Quarter 1, 2026- 2027

Course Name: Coding Practice
Course Code: CSE 422
Student's Full Name: Thai Phan Thanh Sang
Student ID: 2231200114

LAB 02

GitHub Link: https://github.com/thaiphanthanhsang/CSE422_CodingPractice.git

1. PROJECT OVERVIEW

Device Management System is a web application for organizing device categories, devices, and employees. The application is designed with ASP.NET Core MVC (.NET 10), C#, Entity Framework Core, Microsoft SQL Server, Razor Views, and Bootstrap/CSS. It supports CRUD operations and provides device search and filtering. The project follows a simple structure to demonstrate clean code and separation of concerns.

2. PROJECT STRUCTURE AND MAIN CLASSES

Folder/class	Responsibility
Models/	Device, DeviceCategory, Employee, DeviceStatus; properties, relationships, and validation.
Data/	DeviceManagementDbContext: DbSet definitions, unique indexes, and one-to-many relationships.
Controllers/	DeviceCategoriesController, DevicesController, EmployeesController: process requests and coordinate database operations.
Views/	Razor pages (Index, Create, Edit, Delete and optional Details), shared layout, form validation messages.
wwwroot/	Bootstrap resources and custom site.css styling.
Migrations/	Entity Framework Core schema version history for SQL Server.

3. DATA MODEL AND RELATIONSHIPS

DeviceCategory stores Id, Name, and Description. Employee stores Id, FullName, Email, and PhoneNumber. Device stores Id, Name, Code, CategoryId, Status, EntryDate, and an optional EmployeeId. DeviceStatus is an enum with In Use, Broken, and Under Maintenance.

Two one-to-many relationships are defined: one category can contain many devices, and one employee can be assigned many devices. Each device belongs to one category and may be unassigned or assigned to one employee. CategoryId and EmployeeId are foreign keys in Devices.

4. FEATURE IMPLEMENTATION

Category management: MVC actions and Razor forms implement listing, creation, editing, and deletion of categories. A category should not be deleted while devices still reference it.

Device management: Device forms capture device name, unique code, category, status, entry date, and optional employee. Index displays devices and related category/employee information.

Employee management: CRUD views capture full name, email, and phone number; data annotations check required fields and common formatting errors.

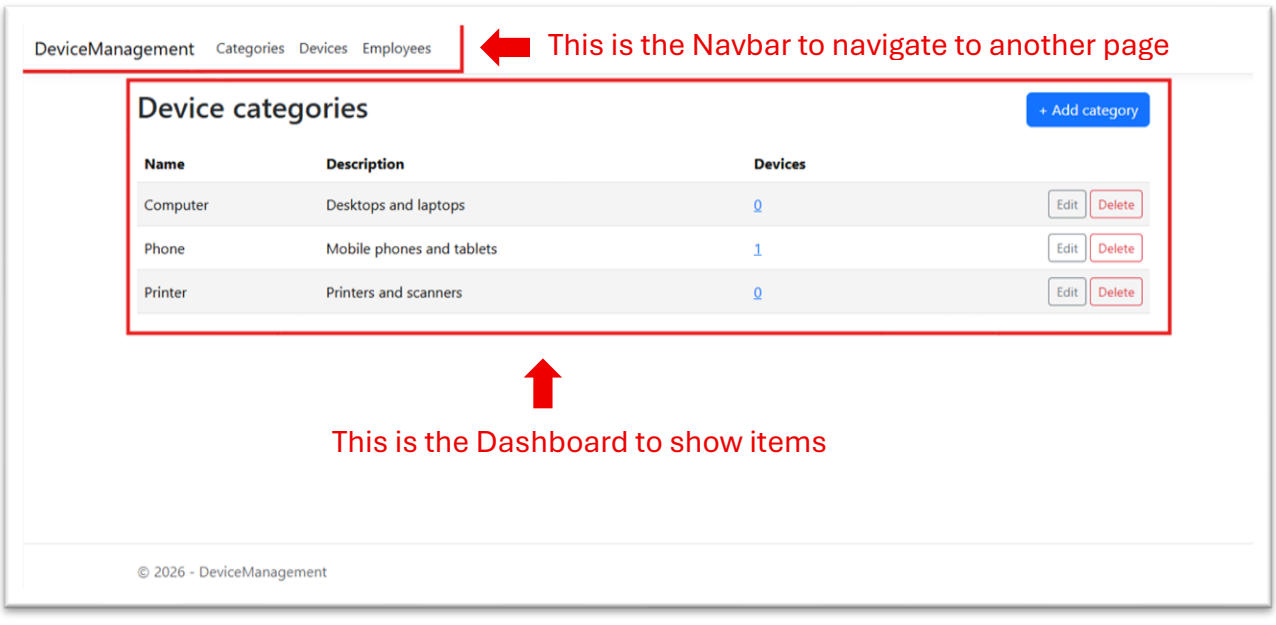
Search and filter: The device list accepts a name/code search term and optional category/status filters. An EF Core query combines the selected conditions before retrieving results.

Validation: Data annotations and ModelState check form input. Unique indexes for device code and employee email provide database-level protection against duplicates.

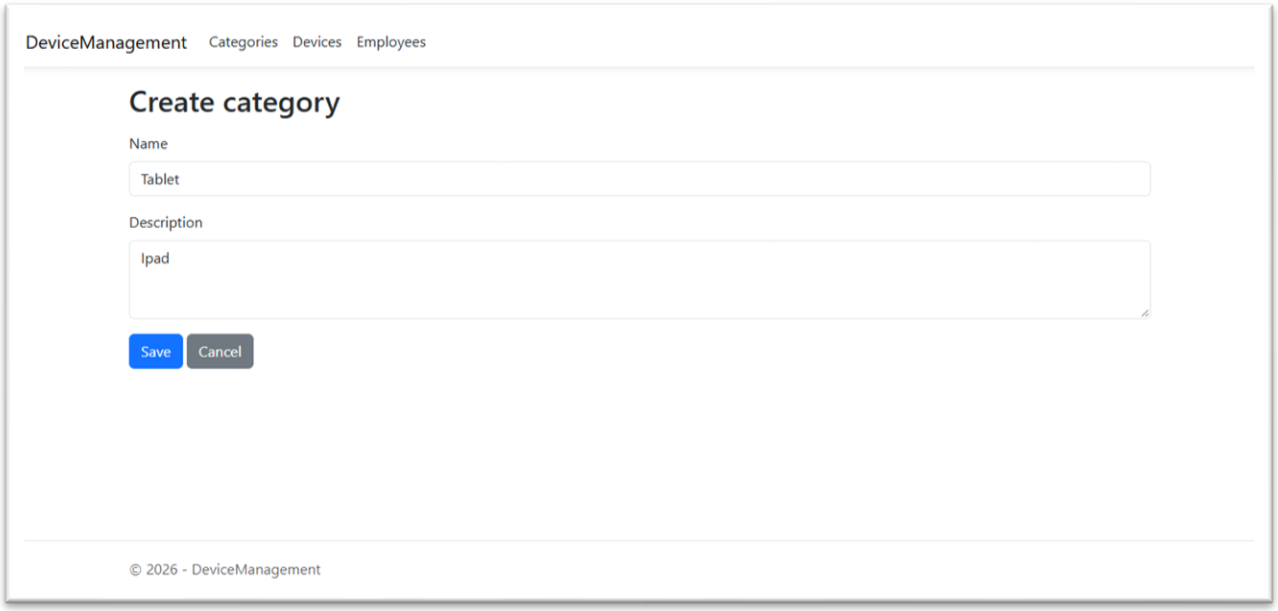
5. CLEAN CODE AND DESIGN DECISIONS

Classes and properties use descriptive PascalCase names, while local variables use camelCase. The MVC pattern separates UI rendering (Views), request handling (Controllers), and data definitions (Models). Dependency injection provides ApplicationDbContext; asynchronous EF Core operations are preferred for database access. DeviceStatus enum avoids inconsistent string values. Shared Razor layout and CSS reduce duplicated interface code.

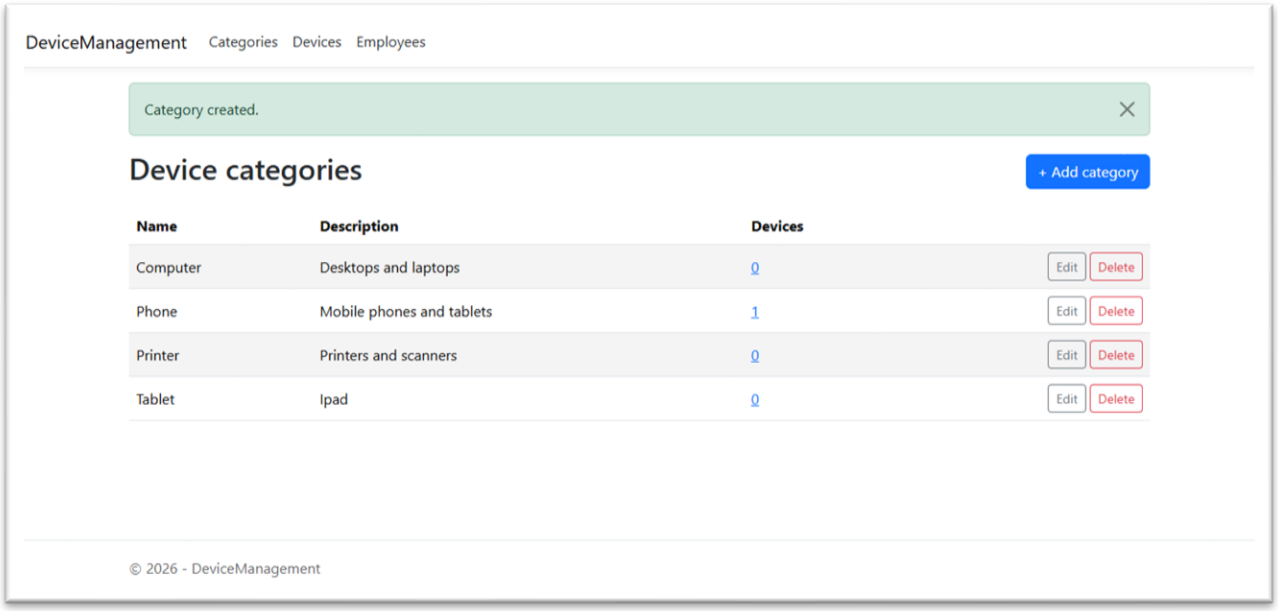
6. DEMO PROJECT



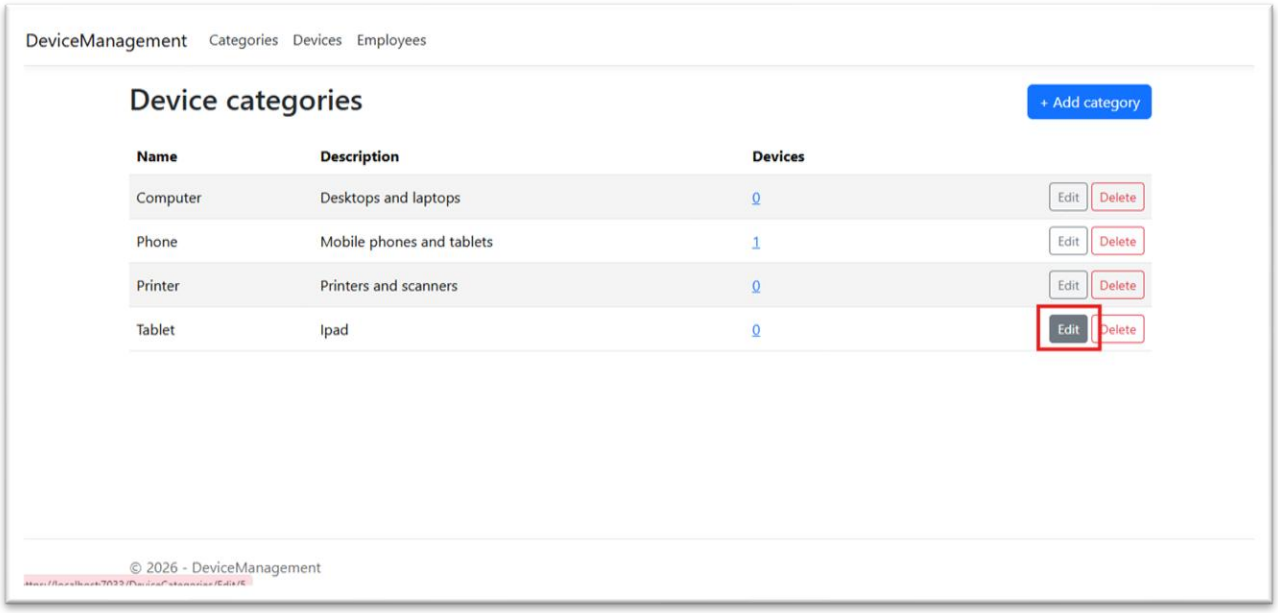
1. Create Page



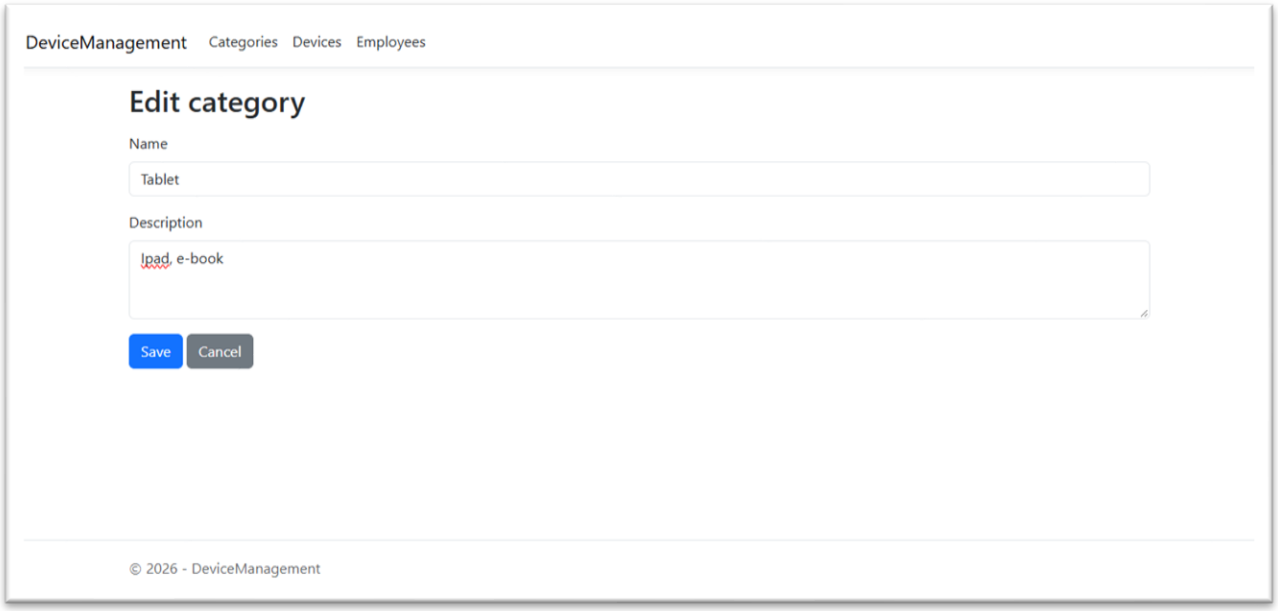
2. After Created



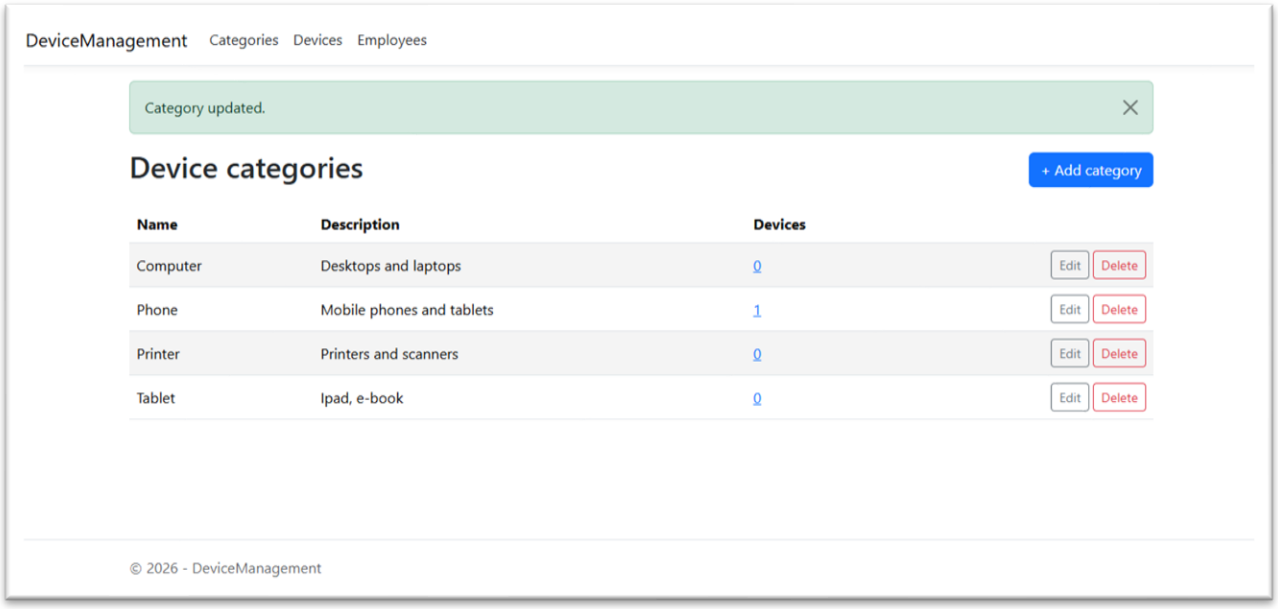
3. Edit Button



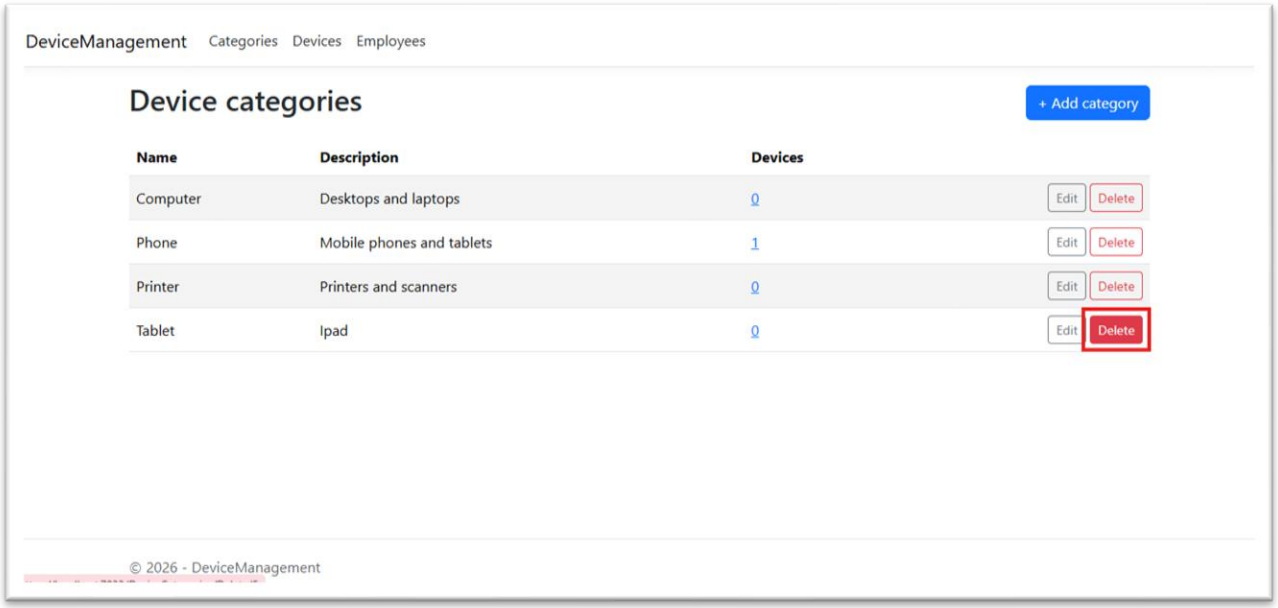
4. Edit Page



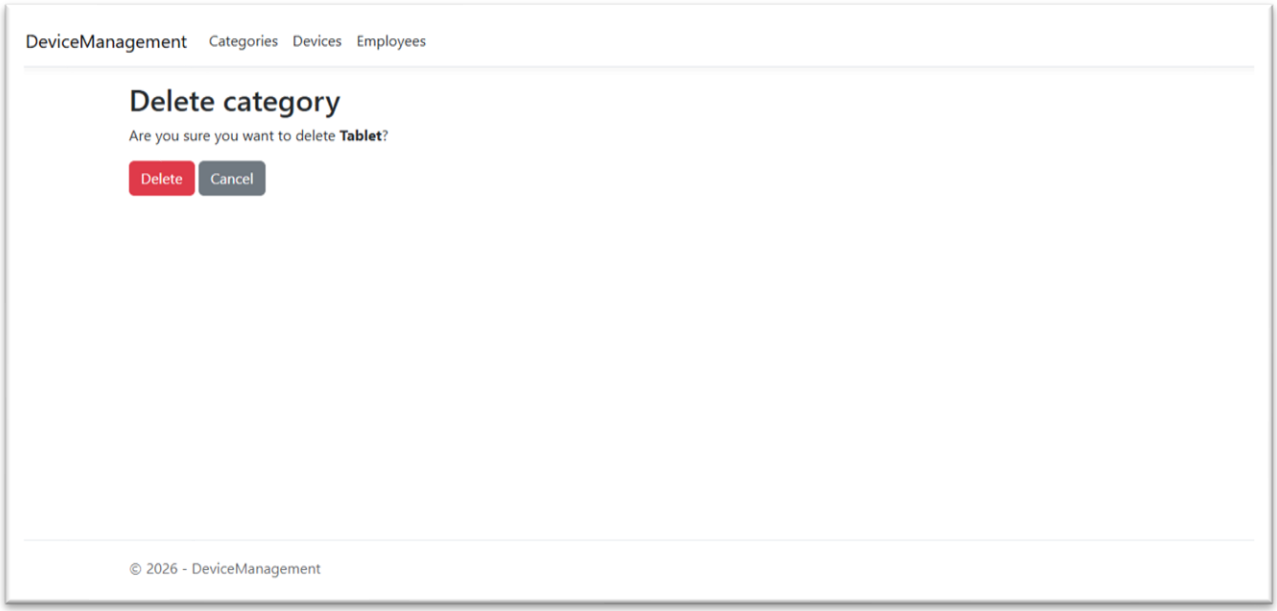
5. After Edited



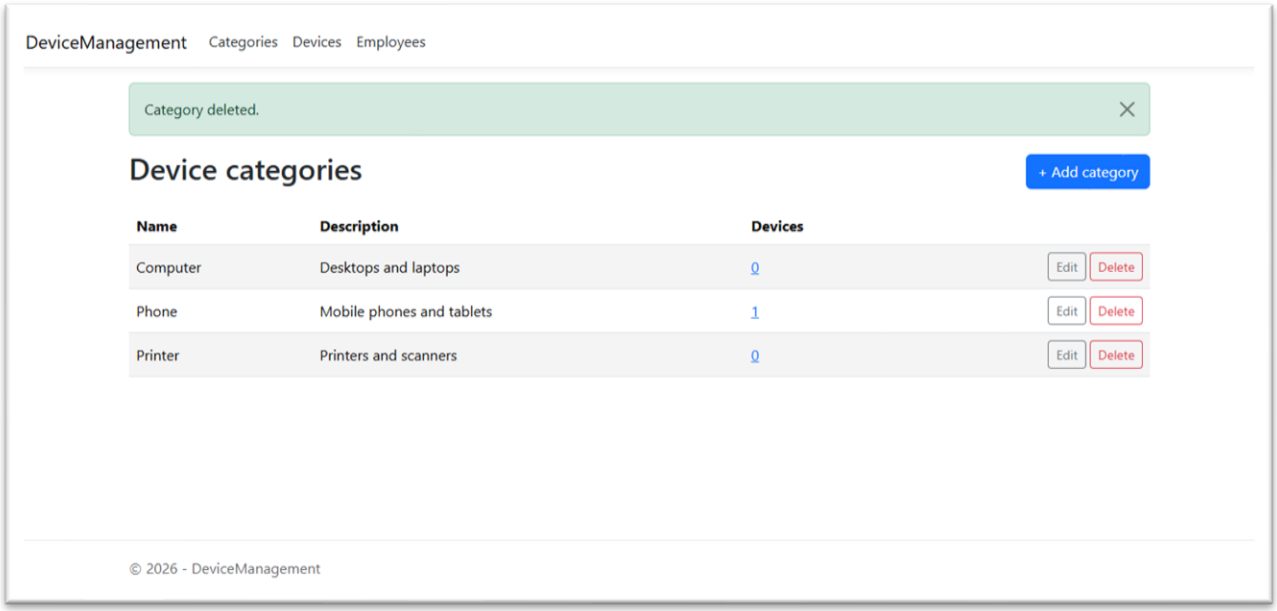
6. Delete Button



7. Delete Page



8. After Deleted



Note: It' s similar for other pages.